

Pre-survey

1. Scan the QR code
2. Select 'Teaching Sign Language with AI' for the workshop you're attending

We encourage you to take the survey; the data will be valuable for our research and for improving future workshops. 😊





Ivana Hernandez, Keren Zhang, Joey Chen



Meet Us

Emu Unicorn Sauce (EUS)

Joey - Software Dev major

Worked on obtaining the required code for the project.

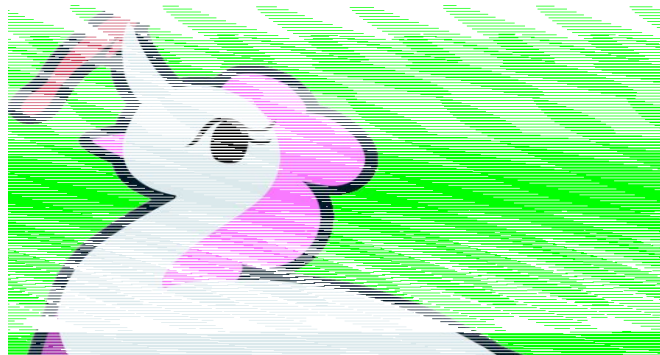
Ivana - Software Dev major

Worked on/trained the AI and the grading system.

Drew the logo for Gesture Control and the EUS mascot

Keren - Data Science major

Worked on integrating the code into a website with both front-end and back-end.



*Emu Unicorn Sauce mascot,
animated by Ivana*



About TAP

- ❖ The *Technology Ambassador Program* (TAP) prioritizes simplifying technology in the view of the common eye, which is why anybody can enroll: from experienced programmers to the technologically illiterate.
- ❖ Students will choose a project to work on during the semester and maintain motivation consistently to progress through all checkpoints.
- ❖ Outside the learning of technical skills needed to work on your project, TAP also leads to developing self-discipline and soft skills, which are arguably just as important!

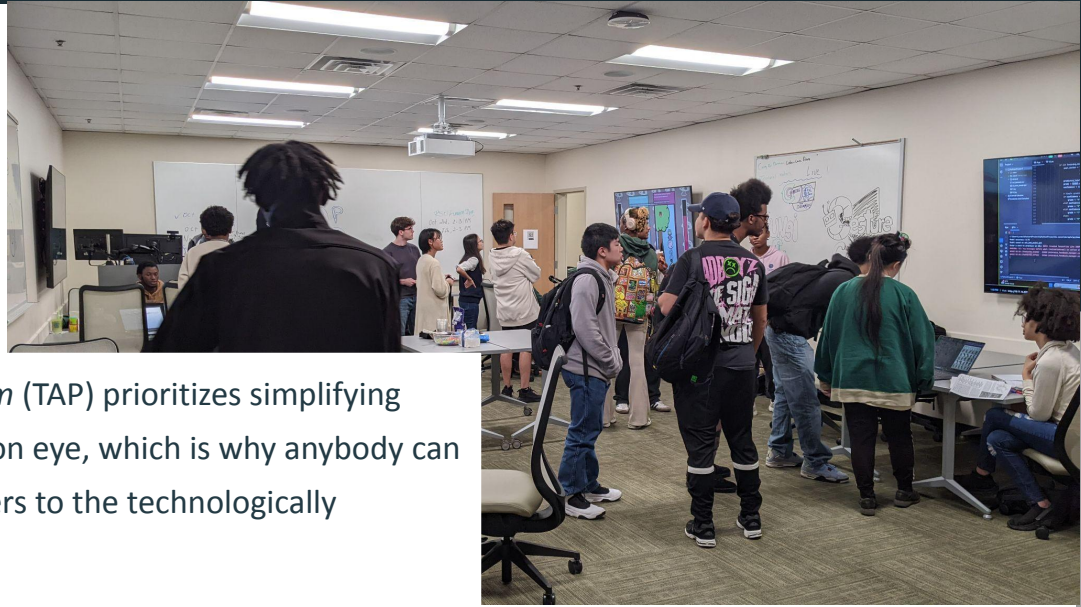


Image from 10/10/25 TAP EXPO



Our Goals

- ❖ Through *Gesture Control*, we can showcase a real-world use of AI through the idea of teaching sign language to the user.
- ❖ Users will learn how to sign a given word by the individual letters in the *American Sign Language* (ASL); the program grades you on your accuracy.
- ❖ AI, especially outside of LLMs, may seem a little alien when compared to other forms of technology. Hopefully we can demystify AI, as this involves basic machine learning that anybody can learn.

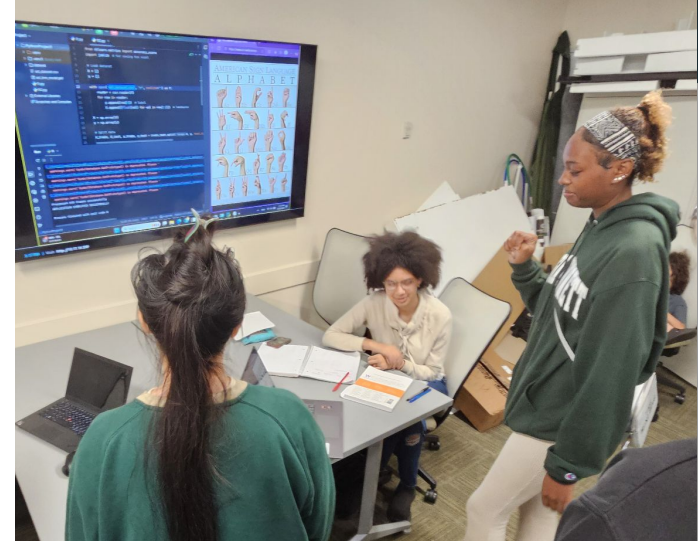


Image from 10/10/25 TAP EXPO



Hand Recognition

- ❖ 'Hand Landmark' code is from MediaPipe, trained off thousands of hands
https://mediapipe-studio.webapps.google.com/studio/demo/hand_landmarker
- ❖ It divides the hand into 21 points, with each point having its own coordinates

The screenshot shows the MediaPipe Studio web interface. At the top, there's a navigation bar with a menu icon, the MediaPipe logo, and the text 'MediaPipe Studio'. Below this is a sidebar with a 'Home' button and a 'VISION' section containing a list of tasks: Object Detection, Image Classification, Image Segmentation, Interactive Segmentation..., Gesture Recognition, Hand Landmark Detection (highlighted with a blue dot), Image Embedding, Face Detection, Face Landmark Detection..., and Pose Landmark Detection.... The main content area is titled 'Hand Landmark Detection' and contains a description: 'Detect hands and their landmarks (key points) in an image or video. You can use this task to localize key points of the hands and render visual effects over them. For more information about how to use the detected hand landmarks and handedness, see the [documentation](#).' Below the description is a light blue box with the heading 'Code examples' and links for 'Android', 'iOS', 'Python', 'Raspberry Pi', and 'Web'. Further down, it says 'The sample parameters below can be changed. See [documentation](#) for more details'. There are three interactive elements: 'Inference delegate:' with a dropdown menu set to 'GPU inference'; 'Model selections:' with a dropdown menu set to 'Hand landmarker'; and 'Demo num hands:' with a slider ranging from 1 to 2, currently set at 2.

MediaPipe Studio

Home

VISION

- Object Detection
- Image Classification
- Image Segmentation
- Interactive Segmentation...
- Gesture Recognition
- Hand Landmark Detec...
- Image Embedding
- Face Detection
- Face Landmark Detect...
- Pose Landmark Detec...

Hand Landmark Detection

Detect hands and their landmarks (key points) in an image or video. You can use this task to localize key points of the hands and render visual effects over them. For more information about how to use the detected hand landmarks and handedness, see the [documentation](#).

Code examples
[Android](#) | [iOS](#) | [Python](#) | [Raspberry Pi](#) | [Web](#)

The sample parameters below can be changed. See [documentation](#) for more details

Inference delegate: GPU inference

Model selections: Hand landmarker

Demo num hands: 1 2



The AI

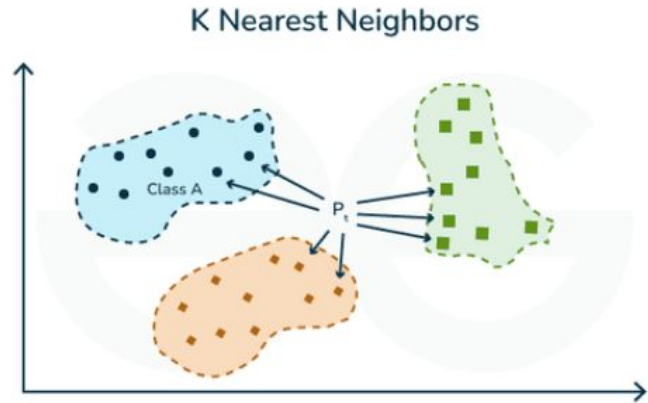
- ❖ Using the MediaPipe code, we had each hand as a set of coordinates to a corresponding letter into a CSV file.
- ❖ 15 samples for each letter was collected (though ideally, you would need hundreds for accuracy)
- ❖ The CSV file was used to train the AI for 'letter recognition'

A	0.5863040	0.5377089	-5.82E-07	0.6650956	0.514521	-0.02117
P	0.631982	0.4389705	1.6512025	0.6545878	0.3721058	-0.02808
P	0.6509067	0.4070387	2.3899721	0.6673443	0.3489736	-0.02832
L	0.5822718	0.6613953	1.2963180	0.650313	0.6372552	-0.02226
E	0.6785417	0.5674809	5.4717407	0.7542836	0.5116733	0.023292



KNN (K-Nearest Neighbors)

- ❖ K-Nearest Neighbors (KNN) is a simple supervised algorithm. In this case, ideal for classification.
- ❖ As the user's hands come into frame, the live coordinates for a given letter are tested against the coordinates in the dataset.
- ❖ Finally, a basic 'if-then' A-D grading system based off how close you were to the predicted letter.

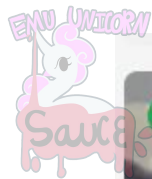




Setting the AI model to run as our backend

- ❖ AI doesn't "see" a picture like humans. It tracks 21 hand **joint points** → analyzes **geometry** → makes a **guess**.
- ❖ Low confidence or bad points = we **ignore** that frame
- ❖ AI is smart, but it also doubts itself. We need to **check confidence** before trusting its answer.

```
[server] Python result: {  
[server]   ok: true,  
[server]   data: {  
[server]     type: 'gesture_result',  
[server]     client_id: 'client_1762135533541_m4qwmoz',  
[server]     hands_detected: true,  
[server]     target: 'A',  
[server]     predicted: 'H',  
[server]     confidence: 0.3333333333333333,  
[server]     landmarks_ok: false,  
[server]     landmarks: [  
[server]       [Object], [Object], [Object],  
[server]       [Object], [Object], [Object],  
[server]       [Object], [Object], [Object],  
[server]       [Object], [Object], [Object],  
[server]       [Object], [Object], [Object],  
[server]       [Object], [Object], [Object],  
[server]       [Object], [Object], [Object]  
[server]     ],  
[server]     server_ts: 1762135692159,  
[server]     inference_ms: 44.9  
[server]   }  
[server] }
```



● LIVE

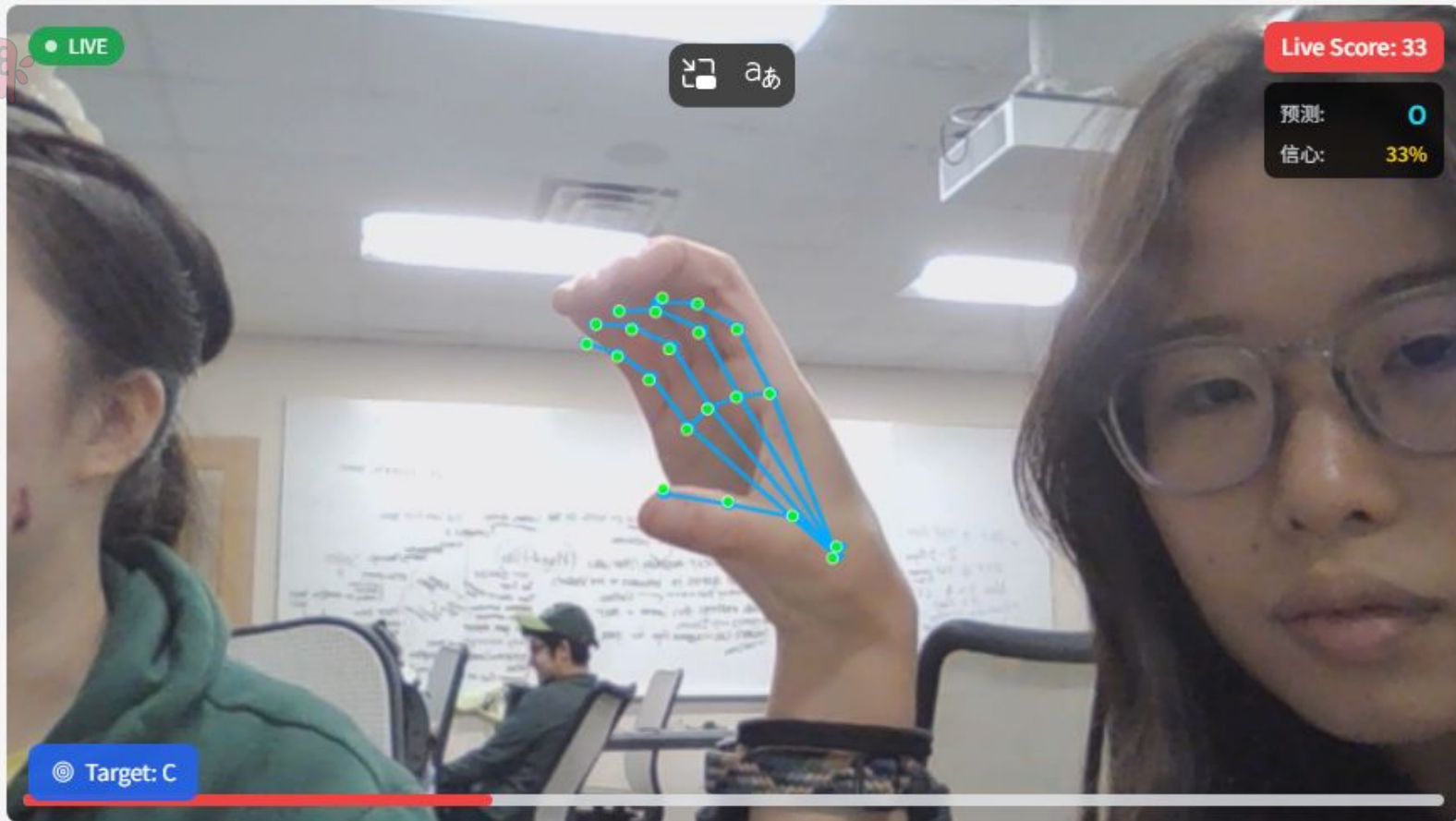


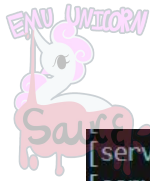
Live Score: 33

予測: 0

信心: 33%

◎ Target: C





Data stored in JSON format = like a excel sheet in coding language

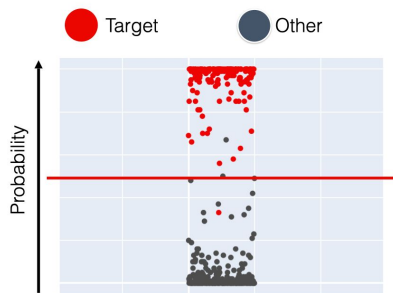
```
[server] python result: {
[server]   ok: true,
[server]   data: {
[server]     type: 'gesture_result',
[server]     client_id: 'client_1762135533541_m4qwm0z',
[server]     hands_detected: true,
[server]     target: 'A',
[server]     predicted: 'H',
[server]     confidence: 0.3333333333333333,
[server]     landmarks_ok: false,
[server]     landmarks: [
[server]       [Object], [Object], [Object],
[server]       [Object], [Object], [Object],
[server]       [Object], [Object], [Object],
[server]       [Object], [Object], [Object],
[server]       [Object], [Object], [Object],
[server]       [Object], [Object], [Object],
[server]       [Object], [Object], [Object]
[server]     ],
[server]     server_ts: 1762135692159,
[server]     inference_ms: 44.9
[server]   }
[server] }
```

Field	Meaning
predicted: "H"	AI's guess for the gesture
target: "A"	The gesture we wanted it to detect
confidence: 0.33	AI's confidence level (0–1). 0.33 = "I'm not sure ..."
landmarks_ok: false	Hand points were not clear → don't trust this frame
inference_ms: 44.9	Time AI needed to think (speed / performance)
landmarks: [...]	21 hand key points (AI sees bones , not skin)



How We Use the AI Output (Decision Logic)

Probability > 50%



Probability > 80%



To make the system stable and fair, we **don't trust every frame**.

We **check three things** before showing a result:

- 1) **Quality** first (Visibility ≥ 0.4 : **Camera sees** the hand clearly)
If the hand keypoints are unclear $\rightarrow$ ignore this frame.

No clean signal = no decision.

- 2) **Confidence** threshold

If the confidence < 0.6 , we don't show it.

Low confidence = "AI **isn't sure** yet."

- 3) **Time** stability (5-frame vote: Avoid random guesses)
The **same gesture must appear across multiple frames** (ex: 5 frames)

One lucky guess $\neq$ real answer.



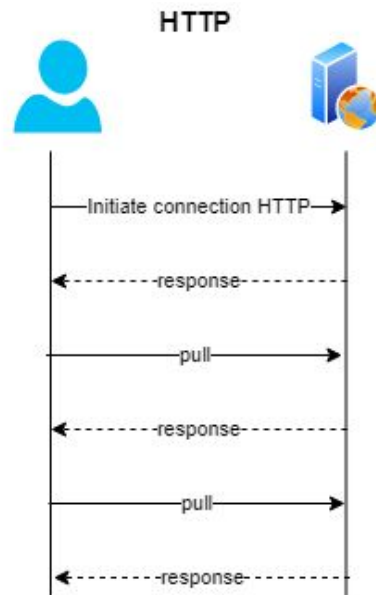
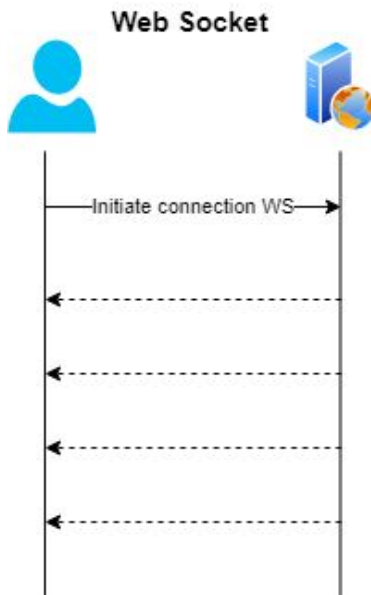
Human + AI communications: WebSocket or HTTP

HTTP (OLD telephone)

One-time request → **one-time response**

Connection **closes after each** message

Good for: forms, loading pages, downloading files





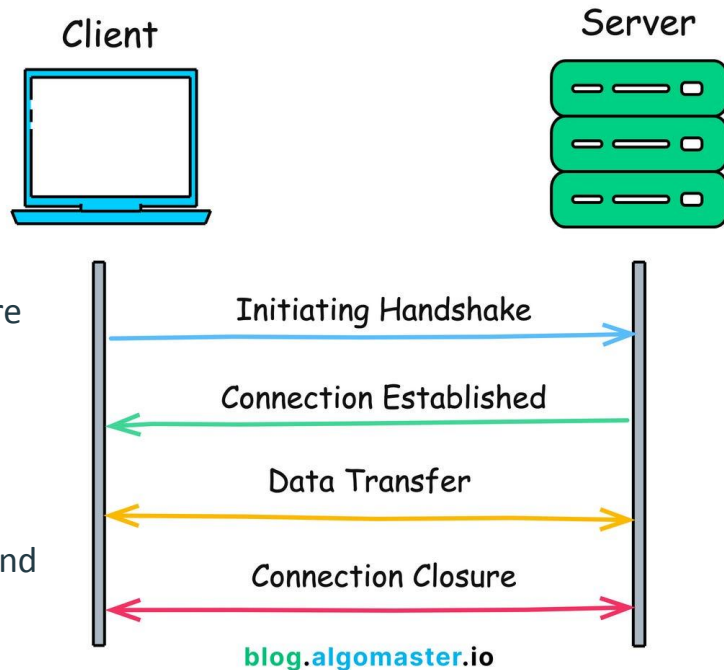
WebSockets

WebSocket(iPhone)

- ❖ Client & server connect once → **stay connected**
- ❖ Can send **messages both directions anytime**
- ❖ Good for: real-time apps (games, video chat, gesture tracking, collaboration)

Our project:

- ❖ Camera sends many hand-tracking frames per second
→ We need **continuous, fast communication** →
WebSocket is the better choice





CORS Explained for Beginners

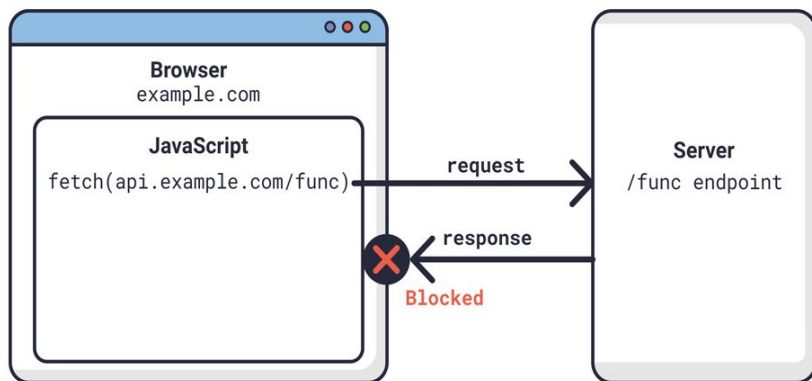


Image from hapnaxy.com

Cross-Origin (CORS) - Why WebSocket Sometimes Gets Blocked

Even if your front-end and back-end are both on Render, they have **different URLs**, so they count as being from “**different places.**”

The browser says:

“Are you sure you’re allowed to connect?”

To fix this, your server must say:

“Yes, I allow connections from this front-end!”

This is called CORS — a **browser safety rule.**



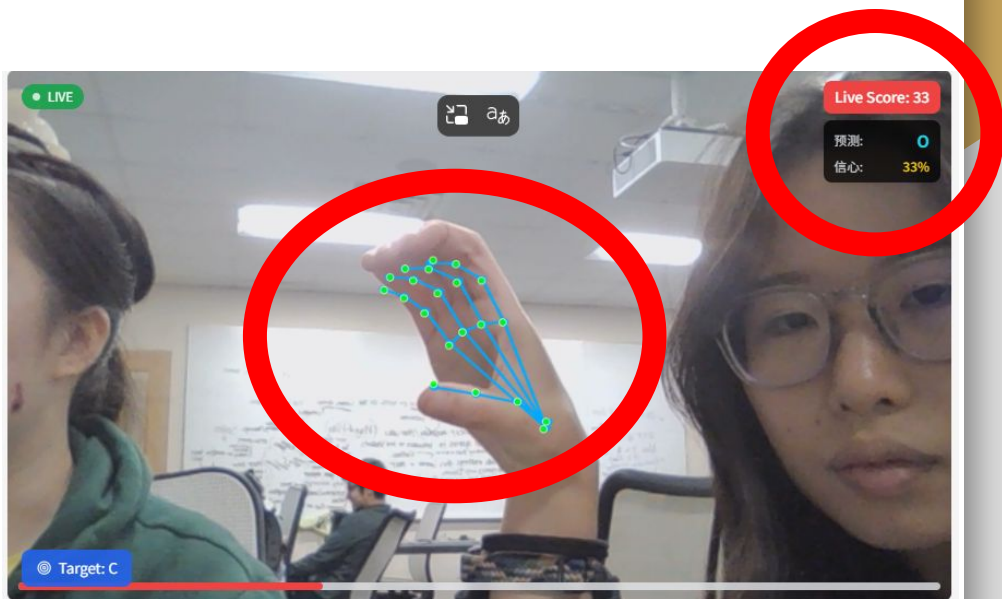
Try Out Our Project!



Head to the Website

<https://gesture-control-xli6.onrender.com>

1. At the drop-down, select 'Webcam'
2. Click 'Start Camera'
3. Use the ASL alphabet picture while playing with the camera.
4. Have Fun! Try A to E. 😊



Post-survey

1. Scan the QR code
2. Select 'Teaching Sign Language with AI' for the workshop you're attending

We encourage you to take the survey; the data will be valuable for our research and for improving future workshops. 😊





Thanks for coming!

Any questions?

